Reasoning Around Recursion

Grounded arithmetic: where it came from, and where it is

Omar

Indian Institute of Science

joint work with the Grounded Deduction group, EPFL DEDIS

September 6, 2026

Two programs

C

```
main() { printf("Hi!"); }
```

```
main() { main(); }
```

Both compile.

Coq / Lean / Agda / Isabelle

```
Fixpoint loop (n:nat) := loop n.
```

Rejected.

Why?

And again, with datatypes

Haskell

```
data Expr
  = Base
  | App Expr Expr
  | Abs (Expr -> Expr)
```

Compiles.

Coq

```
Inductive Expr :=
  | Base :  Expr
  | App :  Expr -> Expr -> Expr
  | Abs :  (Expr -> Expr) -> Expr.
```

Rejected.

And again, with datatypes

Haskell

```
data Expr
  = Base
  | App Expr Expr
  | Abs (Expr -> Expr)
```

Compiles.

Coq

```
Inductive Expr :=
  | Base :  Expr
  | App :  Expr -> Expr -> Expr
  | Abs :  (Expr -> Expr) -> Expr.
```

Rejected.

Non strictly positive occurrence of “Expr” in “(Expr -> Expr) -> Expr”.

A *different* refusal — not termination, **positivity**.

Not one narrow check: a family of restrictions on what you may define.

The usual answer is wrong

"Termination checking is a conservative engineering choice."

The usual answer is wrong

"Termination checking is a conservative engineering choice."

It is not a choice.

It is the **consistency** of your proof assistant.

The usual answer is wrong

"Termination checking is a conservative engineering choice."

It is not a choice.

It is the **consistency** of your proof assistant.

Working out exactly why took fifteen years,
and ended two research programmes.

Background: why anyone wanted a way out

- **Frege** (1893): mathematics from logic, via extensions of concepts.
- **Russell** (1902): naive comprehension is inconsistent.
  $R = \{x : x \notin x\}$. Is $R \in R$?
- Two repairs, both in wide use by 1930:
  - **Russell** — *type theory*
  - **Zermelo** — *axiomatic set theory*

Background: why anyone wanted a way out

- **Frege** (1893): mathematics from logic, via extensions of concepts.
- **Russell** (1902): naive comprehension is inconsistent.
  $R = \{x : x \notin x\}$. Is $R \in R$?
- Two repairs, both in wide use by 1930:
  - **Russell** — *type theory*
  - **Zermelo** — *axiomatic set theory*

Type theory works by **stratification**: every object gets a level, and a collection may only contain objects of strictly lower level.

What stratification costs

Under stratification, $x\,x$ **is not false — it is ungrammatical.**
A thing may never be applied to itself. That is the whole mechanism.

What stratification costs

Under stratification, $x\,x$ **is not false — it is ungrammatical.**
A thing may never be applied to itself. That is the whole mechanism.

But self-application is exactly how you get **recursion**:
from $x\,x$ you can build a fixed-point combinator, and a fixed point solves *any* recursive equation.

What stratification costs

Under stratification, $x\,x$ **is not false — it is ungrammatical.**
A thing may never be applied to itself. That is the whole mechanism.

But self-application is exactly how you get **recursion**:
from $x\,x$ you can build a fixed-point combinator, and a fixed point solves *any* recursive equation.

Ban it, and you lose general recursion.

What stratification costs

Under stratification, $x\,x$ **is not false — it is ungrammatical.**
A thing may never be applied to itself. That is the whole mechanism.

But self-application is exactly how you get **recursion**:
from $x\,x$ you can build a fixed-point combinator, and a fixed point solves *any* recursive equation.

Ban it, and you lose general recursion.

> *"Rather than adopt the method of Russell . . . or that of Zermelo,* ***both of which appear somewhat artificial****, we introduce . . . a certain restriction on the law of excluded middle."*
>
> — Church 1932

Church and Curry: functions, not sets

"Both of them set out to base logic and mathematics on **functions** instead of on set theory."
— Seldin

Extensional — set theory

A function *is* its graph: the set of its input–output pairs. Two functions are the same iff they agree on every input.

So $\lambda n.\, 2n$ and $\lambda n.\, n + n$ are the *same* function.

Intensional — Church, Curry

A function is a *rule*: a finite description of how to get the output. Two functions are the same iff the descriptions are.

So $\lambda n.\, 2n$ and $\lambda n.\, n + n$ are *different* functions.

Church and Curry: functions, not sets

"Both of them set out to base logic and mathematics on **functions** instead of on set theory."
— Seldin

Extensional — set theory

A function *is* its graph: the set of its input–output pairs. Two functions are the same iff they agree on every input.

So $\lambda n.\, 2n$ and $\lambda n.\, n + n$ are the *same* function.

Intensional — Church, Curry

A function is a *rule*: a finite description of how to get the output. Two functions are the same iff the descriptions are.

So $\lambda n.\, 2n$ and $\lambda n.\, n + n$ are *different* functions.

A rule can be applied to a rule. Compilers compile compilers.
No stratification required.

Church, 1932

A Set of Postulates for the Foundation of Logic

- λ as a *logical* primitive, alongside $\Pi, \Sigma, \&, \sim, \iota, A$
- No free variables
- **Restricted excluded middle**: for some arguments $\mathbf{X}$, $\mathbf{F}(\mathbf{X})$ "is undefined and represents nothing"
- Keeps double negation; **abandons *reductio*** in certain cases
- Guarded quantifier: $\Pi(F, G)$ licenses $G(M)$ only once $F(M)$ is *known true*

Church, 1932

A Set of Postulates for the Foundation of Logic

- λ as a *logical* primitive, alongside $\Pi, \Sigma, \&, \sim, \iota, A$
- No free variables
- **Restricted excluded middle**: for some arguments $\mathbf{X}$, $\mathbf{F}(\mathbf{X})$ "is undefined and represents nothing"
- Keeps double negation; **abandons *reductio*** in certain cases
- Guarded quantifier: $\Pi(F, G)$ licenses $G(M)$ only once $F(M)$ is *known true*

Reject LEM · weaken *reductio* · let well-formed terms denote nothing
— a recognisable sketch of GA, ninety-four years early.

Church did not claim consistency

"Whether the system of logic which results from our postulates is adequate for the development of mathematics, and ***whether it is wholly free from contradiction****, are questions which we cannot now answer except by* ***conjecture****."*

— Church 1932, p. 348

Church did not claim consistency

> *"Whether the system of logic which results from our postulates is adequate for the development of mathematics, and* ***whether it is wholly free from contradiction****, are questions which we cannot now answer except by* ***conjecture****."*
>
> — Church 1932, p. 348

He proposed to find out *empirically*.

Curry, 1930

Grundlagen der kombinatorischen Logik

- Started (1922) from a different question: **substitution** in *Principia* is far more complicated than modus ponens — break it into simpler rules
- Combinators $\mathsf{I}, \mathsf{B}, \mathsf{C}, \mathsf{W}, \mathsf{K}, \mathsf{S}$; no bound variables at all
- Consistency proof for the *combinator* part — but no logical axioms given; that half was left for future work

Curry, 1930

Grundlagen der kombinatorischen Logik

- Started (1922) from a different question: **substitution** in *Principia* is far more complicated than modus ponens — break it into simpler rules
- Combinators $\mathsf{I}, \mathsf{B}, \mathsf{C}, \mathsf{W}, \mathsf{K}, \mathsf{S}$; no bound variables at all
- Consistency proof for the *combinator* part — but no logical axioms given; that half was left for future work

On the Russell term $[x](\neg(xx))$: Curry proposed to find postulates from which **it could not be proved that its self-application is a proposition** — while insisting the term *exist* in the system.

Kleene & Rosser, 1935

Both systems are strong enough to enumerate their own provably-total functions. That is enough to formalise **Richard's paradox**.

> *"We are sorry to say that we have a proof of the inconsistency of your system of formal logic."*

Kleene & Rosser, 1935

Both systems are strong enough to enumerate their own provably-total functions. That is enough to formalise **Richard's paradox**.

> *"We are sorry to say that we have a proof of the inconsistency of your system of formal logic."*
>
> *"I am afraid that I can not share your grief . . . you are not sorry but you are full of that profound satisfaction which comes from doing something worth while . . .* ***I hasten to congratulate you both.****"*
>
> — Curry to Kleene and Rosser, 19 November 1934

That — but not *why*

Kleene and Rosser showed the systems were inconsistent.
They did not isolate **which principle** was responsible.

- The proof was long, and specific to those systems
- Curry's response: a 60-page paper (1941) studying the paradox "so as to lay bare its central nerve"
- Church's response (1933): delete four postulates, the ones carrying *reductio* — **it did not save the system**

That — but not *why*

Kleene and Rosser showed the systems were inconsistent.
They did not isolate **which principle** was responsible.

- The proof was long, and specific to those systems
- Curry's response: a 60-page paper (1941) studying the paradox "so as to lay bare its central nerve"
- Church's response (1933): delete four postulates, the ones carrying *reductio* — **it did not save the system**

The answer came in 1942, in three pages.

Curry, 1942 — three pages

The Inconsistency of Certain Formal Logics, JSL 7(3):115–117

Suppose a system has:

1. λ-abstraction that computes (combinatory completeness)
2. $\vdash M \supset M$
3. **contraction**: from $\vdash M \supset (M \supset N)$ infer $\vdash M \supset N$
4. modus ponens

Curry, 1942 — three pages

The Inconsistency of Certain Formal Logics, JSL 7(3):115–117

Suppose a system has:

1. λ-abstraction that computes (combinatory completeness)
2. $\vdash M \supset M$
3. **contraction**: from $\vdash M \supset (M \supset N)$ infer $\vdash M \supset N$
4. modus ponens

Then **every term is provable.**

The derivation

Let $\mathfrak{A}$ be a term with $\mathfrak{A} \equiv (\mathfrak{A} \supset \mathfrak{B})$, obtained from a fixed point of $\lambda X.\, X \supset \mathfrak{B}$.

(1)	$\vdash \mathfrak{A} \supset \mathfrak{A}$	identity
(2)	$\vdash \mathfrak{A} \supset .\, \mathfrak{A} \supset \mathfrak{B}$	unfold the definition
(3)	$\vdash \mathfrak{A} \supset \mathfrak{B}$	contraction
(4)	$\vdash \mathfrak{A}$	fold: (3) *is* $\mathfrak{A}$
(5)	$\vdash \mathfrak{B}$	modus ponens

$\mathfrak{B}$ was arbitrary.

What is *not* in that proof

No negation.

What is *not* in that proof

No negation.

Identity, modus ponens, contraction, unfolding a definition.

Every one is valid **intuitionistically**. Every one is valid in **minimal logic**.

What is *not* in that proof

No negation.

Identity, modus ponens, contraction, unfolding a definition.

Every one is valid **intuitionistically**. Every one is valid in **minimal logic**.

Church had restricted excluded middle.
Church had abandoned *reductio*.

Neither was ever going to be enough.

Curry's own diagnosis

> *"Before he discovered this paradox, Curry had assumed that the contradiction was caused by his postulates for the universal quantifier, but this paradox made it clear that* ***the problem was the postulates for implication alone****."*
>
> — Seldin, *The Logic of Curry and Church*

Curry's own diagnosis

"Before he discovered this paradox, Curry had assumed that the contradiction was caused by his postulates for the universal quantifier, but this paradox made it clear that ***the problem was the postulates for implication alone****."*

— Seldin, *The Logic of Curry and Church*

Church guarded the *quantifier*. Curry blamed the *quantifier*.

It was implication.

What the proof actually needs

Curry states hypothesis 1 as **combinatory completeness**: for any term X in variables $x_1, \ldots, x_n$, there is a closed term A with

$$A\, x_1 \cdots x_n \;=\; X$$

That is: *abstraction is definable*. A is what we would write $\lambda x_1 \ldots x_n.\, X$.
He assumes it because that is what his systems offered.

What the proof actually needs

Curry states hypothesis 1 as **combinatory completeness**: for any term X in variables $x_1, \ldots, x_n$, there is a closed term A with

$$A\, x_1 \cdots x_n \;=\; X$$

That is: *abstraction is definable*. A is what we would write $\lambda x_1 \ldots x_n.\, X$.
He assumes it because that is what his systems offered.
But the proof consumes only *one* term:

$$\mathfrak{A} \qquad \text{with} \qquad \mathfrak{A} \;\equiv\; \mathfrak{A} \supset \mathfrak{B}$$

Abstraction supplies it, via a fixed-point combinator. So does a plain definitional mechanism: write $d \equiv d \to \mathfrak{B}$.

What the proof actually needs

Curry states hypothesis 1 as **combinatory completeness**: for any term X in variables $x_1, \ldots, x_n$, there is a closed term A with

$$A\, x_1 \cdots x_n \;=\; X$$

That is: *abstraction is definable*. A is what we would write $\lambda x_1 \ldots x_n.\, X$.
He assumes it because that is what his systems offered.
But the proof consumes only *one* term:

$$\mathfrak{A} \qquad \text{with} \qquad \mathfrak{A} \;\equiv\; \mathfrak{A} \supset \mathfrak{B}$$

Abstraction supplies it, via a fixed-point combinator. So does a plain definitional mechanism: write $d \equiv d \to \mathfrak{B}$.

The real hypothesis is:

the system admits self-referential definitions.

Combinatory completeness is one route to that — not the only one, and not equivalent to it.

The shape of the result

Unrestricted recursion + ordinary implication

= proves everything.

The shape of the result

Unrestricted recursion + ordinary implication

= proves everything.

And that is why `Fixpoint loop (n:nat) := loop n.` is rejected.

The shape of the result

Unrestricted recursion + ordinary implication
= proves everything.

And that is why `Fixpoint loop (n:nat) := loop n.` is rejected.

Not conservatism. If your system admitted it, you could derive $\bot$.

After 1935

Church, Kleene and Rosser extracted the *pure* λ-calculus — terms and conversion, no logic — and Church retreated to simple type theory in 1940.

Curry did not retreat. He kept the untyped terms and added a predicate H for "is a proposition," using it to guard the logical rules.

After 1935

Church, Kleene and Rosser extracted the *pure* λ-calculus — terms and conversion, no logic — and Church retreated to simple type theory in 1940.

Curry did not retreat. He kept the untyped terms and added a predicate H for "is a proposition," using it to guard the logical rules.

Everything you use descends from Church 1940.

GA is closer to Curry's road.

Where everyone landed

	unrestricted recursion	ordinary rules	non-triviality
Church 1932 · Curry 1930	kept	weakened neg.	**lost**
STLC · Coq · Agda · Lean · HOL	**dropped**	kept	kept
Contraction-free (Fitch, Grišin, Cantini)	kept	**structural**	kept
Paraconsistent (Priest)	kept	**no explosion**	**contained**
Kripke 1975 (truth gaps)	kept	**partial**	kept
Grounded arithmetic	**kept**	**preconditions**	**kept**

Everyone since 1942 kept implication and gave up recursion.

Kripke, 1975 — there is no sieve

Whether a sentence is paradoxical can depend on *empirical facts*, not on its shape.

Kripke, 1975 — there is no sieve

Whether a sentence is paradoxical can depend on *empirical facts*, not on its shape.

> *"The moral: an adequate theory must allow our statements involving the notion of truth to be* ***risky*** *. . . There can be* ***no syntactic or semantic "sieve"*** *that will winnow out the "bad" cases while preserving the "good" ones."*
>
> — p. 692

Kripke, 1975 — there is no sieve

Whether a sentence is paradoxical can depend on *empirical facts*, not on its shape.

> *"The moral: an adequate theory must allow our statements involving the notion of truth to be* ***risky*** *. . . There can be* ***no syntactic or semantic "sieve"*** *that will winnow out the "bad" cases while preserving the "good" ones."*
>
> — p. 692

So you cannot fix this by restricting *what may be written*.
Exactly the argument against the type-theoretic route.

Grounded

> *"If ultimately this process terminates in sentences not mentioning the concept of truth, so that the truth value of the original statement can be ascertained, we call the original sentence* ***grounded****; otherwise,* ***ungrounded****."*
>
> — p. 694

Grounded

> *"If ultimately this process terminates in sentences not mentioning the concept of truth, so that the truth value of the original statement can be ascertained, we call the original sentence **grounded**; otherwise, **ungrounded**."*
>
> — p. 694

The detail people miss — the **truth-teller**:

(3) "(3) is true."

"which, though not paradoxical, yield no determinate truth conditions" (p. 693); "Sentences such as (3), though not paradoxical, are ungrounded" (p. 694).

Grounded

> *"If ultimately this process terminates in sentences not mentioning the concept of truth, so that the truth value of the original statement can be ascertained, we call the original sentence **grounded**; otherwise, **ungrounded**."*
>
> — p. 694

The detail people miss — the **truth-teller**:

(3) "(3) is true."

"which, though not paradoxical, yield no determinate truth conditions" (p. 693); "Sentences such as (3), though not paradoxical, are ungrounded" (p. 694).

Groundedness is not about *paradox*.

It is about having a basis.

The least fixed point

- $T(x)$ is interpreted by a **partial set** (S_1, S_2) — extension, antiextension, undefined outside $S_1 \cup S_2$
- Connectives evaluate by **Kleene's strong three-valued scheme** (p. 700; fn. 18 cites Kleene 1952, §64)
- The jump ϕ is **monotone**: extending T never changes a value already established. "This property — technically, the monotonicity of ϕ — is crucial for all our constructions." (p. 703)
- Iterate transfinitely $\Rightarrow$ a **minimal** fixed point: "any fixed point extends" it (p. 705)

The least fixed point

- $T(x)$ is interpreted by a **partial set** (S_1, S_2) — extension, antiextension, undefined outside $S_1 \cup S_2$
- Connectives evaluate by **Kleene's strong three-valued scheme** (p. 700; fn. 18 cites Kleene 1952, §64)
- The jump ϕ is **monotone**: extending T never changes a value already established. "This property — technically, the monotonicity of ϕ — is crucial for all our constructions." (p. 703)
- Iterate transfinitely $\Rightarrow$ a **minimal** fixed point: "any fixed point extends" it (p. 705)

"(Being a fixed point, $\mathcal{L}_\sigma$ is a language that **contains its own truth predicate**.)" — p. 705

Kripke's caution: "*Undefined* is not an extra truth value" (fn. 18) — this is not three-valued logic.

What GA takes, and what it adds

From Kripke

The word *grounded*.

Ungrounded expressions denote nothing, so paradoxes are inert.

A language containing its own truth predicate.

GA adds

Grounding in *computation*, not a semantic construction.

The condition as *preconditions on inference rules*.

Arithmetic, Turing-complete computation, machine-checked metatheory.

What GA takes, and what it adds

From Kripke

The word *grounded*.

Ungrounded expressions denote nothing, so paradoxes are inert.

A language containing its own truth predicate.

GA adds

Grounding in *computation*, not a semantic construction.

The condition as *preconditions on inference rules*.

Arithmetic, Turing-complete computation, machine-checked metatheory.

BGA proves its own truth predicate for grounded sentences (Thm. 4.20) — Kripke's *raison d'être*, in a system that also computes.

Habeas quid — "have a thing"

You may not reason with a term until you have shown it *has a value*.

$$\frac{\Gamma \vdash p\ \mathsf{B} \qquad \Gamma \cup \{p\} \vdash q}{\Gamma \vdash p \to q}\ (\to I) \qquad \frac{\Gamma \vdash p\ \mathsf{B} \quad \Gamma \cup \{\neg p\} \vdash q \quad \Gamma \cup \{\neg p\} \vdash \neg q}{\Gamma \vdash p}$$

$p\ \mathsf{B} \equiv p \vee \neg p$ — classically a tautology, so the preconditions are free and the rules collapse to the familiar ones.

In GA it is a **dynamic type check**: a demand that p terminate with a boolean.

In Curry's terms: GA satisfies hypotheses **1, 2 and 4**.
It breaks **3**.

Both paradoxes die of malnutrition

Liar $L \equiv \neg L$

Admitted as a definition.

Refutation by contradiction needs L B, which needs the refutation.

No value. Nothing to explode from.

Curry $C \equiv C \to P$

Admitted as a definition.

Step (3) now needs C B first, obtainable only via the implication we are trying to introduce.

Circular obligation. No proof.

The paradoxes are **expressible** and simply not **provable**.

Does GA forbid self-application?

No.

Does GA forbid self-application?

No.

PGA has higher-order application $t[t]$ as a *primitive*. So $f[f]$ is a well-formed term.

Everything is a natural number: $f[a]$ reads f as a Gödel code of a primitive-recursive step function, applied to a, with an unbounded search.
$\mathcal{S}, \mathcal{K}, \mathcal{I}$ are just particular numerals. No λ binder — but λ is derivable by combinator conversion.

Does GA forbid self-application?

No.

PGA has higher-order application $t[t]$ as a *primitive*. So $f[f]$ is a well-formed term.

Everything is a natural number: $f[a]$ reads f as a Gödel code of a primitive-recursive step function, applied to a, with an unbounded search.
$\mathcal{S}, \mathcal{K}, \mathcal{I}$ are just particular numerals. No λ binder — but λ is derivable by combinator conversion.

BGA is the interesting contrast: no term is ever applied to a term. Recursion comes only from $d(t, \ldots, t)$ — definition symbols applied to argument terms, freely mutually recursive, no ordering, no obligation at definition time.

And it is Turing-complete anyway.

The restriction is not on what you may write

Type theory's mechanism is **grammatical**: make $x\,x$ ill-formed, and the paradox cannot be stated.

The restriction is not on what you may write

Type theory's mechanism is **grammatical**: make $x\,x$ ill-formed, and the paradox cannot be stated.

GA does the opposite. All of these are **well-formed**:

$$f[f] \qquad L \equiv \neg L \qquad C \equiv C \to P \qquad \texttt{collatz}(n)$$

The restriction is not on what you may write

Type theory's mechanism is **grammatical**: make $x\,x$ ill-formed, and the paradox cannot be stated.

GA does the opposite. All of these are **well-formed**:

$$f[f] \qquad L \equiv \neg L \qquad C \equiv C \to P \qquad \texttt{collatz}(n)$$

The restriction is on what you may **infer**.

Which is why the paradoxes are *expressible* and merely not *provable* — and why a definition never needs a termination proof to be admitted.

Six papers, two years

2024	GD	the idea, and the propositional core
2025	BGA / PGA	machine-checked metatheory
2026	RGA	quantifiers, grounded reflectively
2026	Internalized truth	its own truth predicate, full language
2026	OGA	one more rule — incompleteness becomes *priced*
2026	Grounded set theory	a graded ZF
now	**Isabelle/Pure**	**can anyone actually use it?**

Roughly: *idea* → *proof it works* → *make it a logic* → *make it reflective* → *price the gap* → *make it a foundation* → *make it usable*.

Four of the six are from 2026.

1 · *Reasoning Around Paradox with Grounded Deduction*

- Kripke-inspired: let ungrounded expressions denote nothing
- *habeas quid* as *preconditions on inference rules* — the central move
- An impossibility triangle:
  consistency · unrestricted LEM · unrestricted recursive definitions
- Propositional GD, predicate logic, arithmetic; a computational interpretation and a denotational semantics
- Speculative closing: reflective idealization, "Cantor's paradise regained. . . ?"

Preliminary — no machine-checked metatheory.

2 · *Have a thing?* — BGA and PGA

Two concrete systems, everything checked in Isabelle/HOL.

BGA — minimal kernel

syntactic consistency
semantic soundness
semantic completeness
represents every general-recursive function
Gödel arithmetization + reflection
its own truth predicate (grounded sentences)

PGA — expressive layer

primitive propositional operators
higher-order functions via SKI
consistency, completeness
reduces to BGA

16 of Copi's 19 classical rules survive *unmodified*

Proof-theoretic strength: somewhere in $[\omega^\omega,\ \omega_1^{\mathrm{CK}}]$ — **open**.

Consistent *and* complete *and* arithmetic?

- Gödel I concerns **syntactic** completeness — for every f, $\vdash f$ or $\vdash \neg f$.
  BGA is emphatically not that, and cannot be: LEM is built into the definition.
- What is proved is **semantic** completeness, *with respect to the operational-semantic model*.
  Every formula that reduces to 1 under all assignments is provable. Not: "GA proves every truth of arithmetic."
- Gödel's argument assumes a classical *object* logic. That assumption fails; the metalogic assumption still holds.

The consistency and completeness proofs are simulation arguments in opposite directions — **the Church–Turing thesis continuing to hold.**

3 · *Computable Quantification in RGA*

Quantifiers — grounded **reflectively**:

a universal is *true* when the system's own proof search certifies its schematic instance,
false when it refutes a particular numeral instance.

- Proves totality of $+$ and $\times$ as internally quantified theorems
- Represents **exactly the recursively enumerable sets**
- Machine-checked: soundness, consistency, open completeness, $\mathbb{N}$-soundness, a Church–Turing characterisation
- **ω-incompleteness** — grounded truth is r.e., so some family has every numeric instance provable while its closure is *semantically ungrounded*

DNE holds · quantified excluded middle fails · refuted universals yield explicit witnesses
the deduction theorem's abstraction direction fails at ungrounded hypotheses

4 · *Internalized Truth in RGA*

Tarski: no consistent *classical* system with arithmetic defines its own truth.

- A truth predicate for RGA's **full language, quantifiers included**, as an *internal RGA term*
- Compiled from a primitive-recursive decider; adequate in both directions
- A square of metatheorems — provable $\Rightarrow$ internally true; grounded-true $\Rightarrow$ internally provable; internal truth $\Rightarrow$ internal provability; consistency follows
- Two *disjoint* internal machines: a certified decider and a certified proof-checker, both RGA terms

Along the way: coded syntax and substitution, symbolic unfolding, internal strong induction — **evidence that RGA supports nontrivial mathematical reasoning.**

5 · *Idealizing Useful Fictions in OGA*

RGA can quantify over its own computations — but **cannot certify that its own unbounded searches have definite yes-or-no answers**.

OGA closes that openness with **exactly one rule**:

ATI, the ω-grounded universal

if every numeric instance of a universal is certified decided,
the universal is certified decided.

Otherwise identical to RGA, symbol for symbol. Every consequence machine-checked.

Certificates become **abundant**: every totality question about a computable function is certified to *have* an answer —
whether or not anyone can produce it.

What that one rule costs, and buys

	RGA	**OGA**
a certificate says	a computation settles it	the question *has* an answer
provability	r.e.	r.e., primitive-recursive checker
truth	**r.e.**	**deliberately *not* r.e.**
the residue	ω-incompleteness	**fictions**

In OGA the Gödel sentence is classified *unconditionally* as a **genuine fiction**: neither provable nor refutable, yet *valued* — and carrying a **computable pedigree** recording exactly what adopting it as an axiom commits you to.

Adoption is itself a theorem suite: extending OGA by any finite stock of true fictions is consistent,
and independently certified adoptions can never collide.

6 · *Fact and Fiction in Grounded Set Theory*

"Classical set theory buys its power on credit:
its axioms assert totalities no computation could survey."

Sets are **formula codes**; membership is *defined* via provability — no primitive $\in$, and no set axioms assumed.

theorems	empty, pairing, separation, foundation, choice, complement, **Tarski universe**
fail	**union, replacement, powerset** — the *collecting* axioms, recursion-theoretically
graded	**extensionality** — pointwise provable, uniformly a Gödel sentence

"ZF fails computably, but holds fictionally,
and the fiction is measurable."

What the programme has, and has not

Settled

Unrestricted recursive definitions, admitted with *no* obligation at definition time

Consistency — machine-checked

Semantic completeness

Quantifiers

Its own truth predicate

Incompleteness *priced* — fictions with pedigrees

A graded set theory

Open

Proof-theoretic strength

Whether *habeas quid* obligations can be discharged *cheaply enough*

Datatypes beyond $\mathbb{N}$

Notation, tactics, automation

Whether anyone can reason in it

Church and Curry were defeated by the *consistency* question.
That one is answered. The usability question is not.

The gap

A logic is not a tool.

- The metatheory is done in **Isabelle/HOL** — which adds HOL's axioms to GD's trusted base, and embeds GD's primitives inside a very different logic
- Good for *studying* GA. Wrong for *reasoning in* GA.
- A foundational system should sit atop a **minimal** framework

Kehrli (2025): formalize GA atop **Isabelle/Pure** instead.

What makes GA hard to use

In classical logic, p B is free — it is the law of excluded middle.

In GA it is **work**.

Every *habeas quid* premise is a proof obligation that a human or a tactic must discharge.

What makes GA hard to use

In classical logic, p B is free — it is the law of excluded middle.

In GA it is **work**.

Every *habeas quid* premise is a proof obligation that a human or a tactic must discharge.

So the engineering question is sharp and checkable:

can these obligations be discharged automatically often enough that ordinary reasoning stays tolerable?

This work

Building a usable theorem prover for grounded deduction on Isabelle/Pure.

- Proof search and tactics for *habeas quid* obligations
- Conditional rewriting where equality is not reflexive
- Induction and case analysis under grounding premises
- Encoding inductive datatypes in a system with only $\mathbb{N}$

[details follow]

Reasoning Around Recursion

Curry proved you cannot have both general recursion
and ordinary implication.

Everyone since has given up recursion.

Grounded arithmetic gives up something in implication instead.